Projeto de Base de Dados

First Submission

Afonso Tavares Fernandes de Almeida Saraiva - up202304461

Sara Marques Ribeiro - up202305327

Context description

The real estate agency operates across various neighborhoods within a city and requires a system to efficiently manage its operations. Its database needs to various.

The agency's operating neighborhoods must have a name and a neighborhoodID. Each property can only be associated with one neighborhood, but the neighborhoods can have several properties linked to them and the location of both.

Geographical data, such as locationID, latitude, and longitude, are held by a specific Location object that is dependent on properties. This object facilitates the accurate determination of each property's exact position.

The system holds information on properties, including propertyID, name, address, type (connected to PropertyType), and value (representing sale or rental values), since properties are the main assets of the real estate company. A property may have several documents tied to it, but it is only connected to one agency and one neighborhood.

The PropertyType entity defines the classification of properties (house, apartment, land). It holds a propertyTypeID and a description to clarify the type of property.

Agencies manage the properties and have an agencyID, name, and address. An agency can handle multiple properties and is responsible for coordinating the activities of its real estate agents.

Agents are employees of the agency who handle transactions. Each agent has an agentID and a transactionsNumber to track the number of transactions they have handled. The relationship between agents and transactions is managed through the AgentTransaction entity, which includes details like the agentRole and the assignedDate for each transaction they handle. The personal details of agents are stored in the Person entity.

The Person generalization stores personal information for agents and clients, with attributes such as personID, name, phoneNumber, and email.

Clients are individuals or organizations that interact with the agency for buying, selling, or renting properties. Each client has a clientID, address, and a type (buyer or seller). Clients can participate in multiple transactions over time and are associated with specific transactions and appointments.

Transactions representing the buying, selling, or renting of properties. Each transaction includes a unique identifier (transactionID), the date it occurred, value, and type (buy, sell, rent). Transactions can involve multiple properties and are linked to a single client. They may also have a corresponding contract.

The TransactionHistory records the changes in status for each transaction, including details like transactionHistoryID, status, and statusDate, to monitor the progression of transactions.

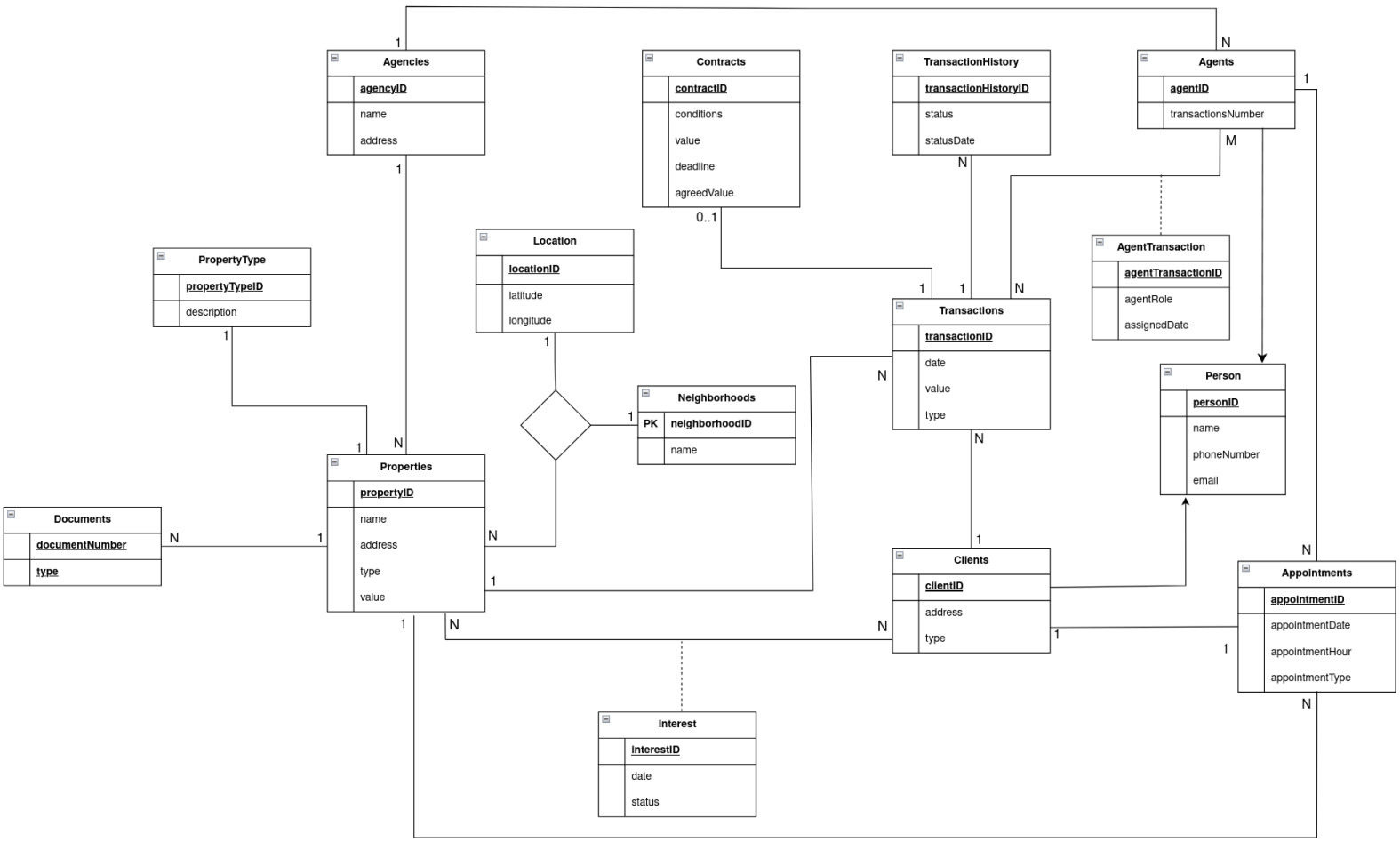
Contracts specify the details of a transaction, including conditions, deadlines, and agreed values. Each contract is stored with a contractID and is linked to a single transaction. However, not every transaction has an associated contract.

Documents are crucial for formalizing transactions and properties. Each document has a documentNumber, type (purchase agreement, lease, or deed), and can be associated with multiple properties.

Appointments help manage property visits, organizing interactions between clients and agents. Each appointment has appointmentID, appointmentDate, appointmentHour, and appointmentType, specifying the purpose of the visit.

The Interest entity keeps a record of clients' interest in specific properties. It includes fields like InterestID, date, and status.

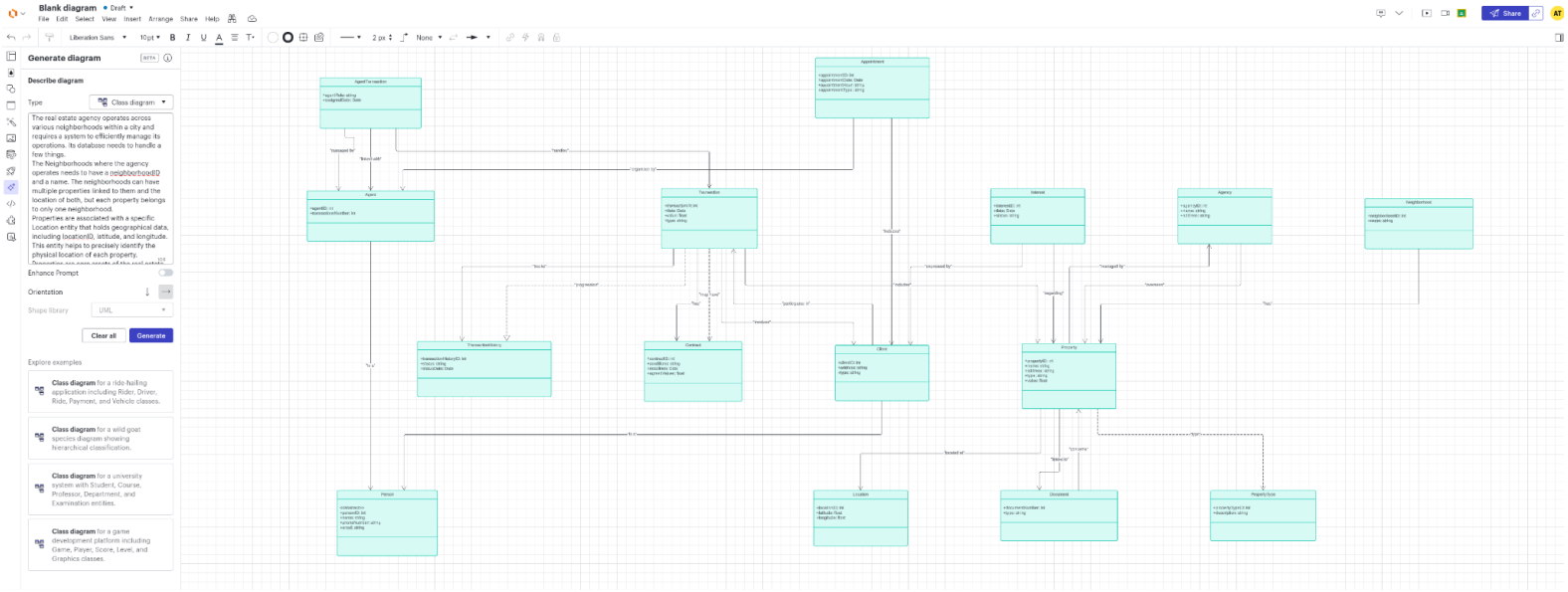
Initial Concept Model



Generative AI Integration

To accomplish this task, we utilized Lucidchart's generative diagrams tool. We accessed the "Generate Diagram" feature located on the right toolbar, chose the "Class Diagram" option, and then provided the initial prompt derived from our context description (excluding the last three paragraphs due to word limit constraints).

First result:



After the first result we gave it to ChatGPT with the prompt “What can you take from this image and do you think you can improve the UML diagram?”.

Second Result:

The image you shared is a UML class diagram representing the database structure for a real estate agency. Here are some observations and potential improvements for the diagram:

Observations from the Diagram:

1. **Entities Included**: The diagram contains entities like `Agent`, `Transaction`, `Appointment`, `Client`, `Contract`, `Property`, `PropertyType`, `Location`, `Neighborhood`, and more, which are logically related to a real estate management system.
2. **Relationships**:

    - There are several relationships between these entities. For example, `Agent` is related to `Transaction`, `Client`, and `Appointment`.
    - `Property` is associated with `Location`, `Neighborhood`, and `PropertyType`.
    - The use of multiplicities (like 1-to-many or many-to-many) in these associations helps define how entities are related.
3. **Attributes**:
    - The attributes listed for each entity seem relevant. For example, the `Property` entity includes fields like `address`, `type`, and `value`, which are standard in a real estate context.
    - Entities like `Transaction` and `Appointment` also have specific details, such as dates and transaction IDs.

Suggestions for Improvement:

1. **Clarify Relationship Lines**:
    - Some relationship lines and connectors in the diagram appear crowded and could be cleaned up for better readability. Organizing them to avoid overlapping and using consistent arrow styles will help clarify the relationships.
2. **Add Multiplicity Details**:
    - Explicitly mention the cardinalities (like 1..*, 0..1, etc.) for all relationships if they are not already specified. This will help define how many instances of one class relate to instances of another class.
3. **Inheritance and Generalization**:
    - If some classes share similar attributes, consider using inheritance. For example, if both `ResidentialProperty` and `CommercialProperty` exist, they could inherit from a base `Property` class.
4. **Consistency in Naming Conventions**:
    - Ensure that the naming conventions for attributes and classes are consistent throughout the diagram (e.g., camelCase or snake_case). This standardization improves the clarity of the model.
5. **Use of Aggregation or Composition**:
    - For entities that strongly depend on other entities, using aggregation (denoted by a diamond symbol) or composition might provide a clearer understanding of the relationship between the entities.
6. **Detailed Data Types**:
    - While some data types are specified, adding more detail where possible (like `string`, `int`, `DateTime`, etc.) can help developers understand the exact nature of the data involved.
7. **Group Related Entities**:
    - Consider visually grouping related entities in distinct sections or using color coding to differentiate different modules, such as "Transactions," "Properties," and "Clients." This makes it easier to navigate the diagram.

We thought it focused too much on “reading” the image instead of proposing new ideas so we gave it another prompt “But what would you change conceptually about the diagram i first gave you”.

Third result:

Conceptually, the original diagram you provided is quite detailed, but it can be refined to make it clearer, more modular, and to better follow UML best practices. Here are some specific conceptual changes I would suggest to improve the design:

1. Simplify Relationships

- **Reduce Complexity in Relationships:** In the original diagram, some relationships are overly complex, leading to cluttered lines and a lack of clarity in how entities interact. Simplifying and organizing these relationships into clear, direct associations would make the diagram more understandable.
- **Use Aggregation and Composition:** Where appropriate, replace plain associations with aggregation or composition to indicate stronger relationships. For example, a `Property` could be composed of a `Location`, as a location is an intrinsic part of the property.

2. Inheritance and Generalization

- **Add Inheritance Where Applicable:** Consider using inheritance to handle common attributes across entities. For example, if there are different types of properties (`ResidentialProperty` and `CommercialProperty`), they could inherit from a base `Property` class to avoid redundancy.
- **Entity Abstraction:** Abstract common elements like `Person` that could be shared by `Agent` and `Client`. If both these classes have similar attributes like `name` and `contactDetails`, a parent `Person` class could hold these shared properties.

3. Modular Design with Clear Grouping

- **Separate Modules:** Group related entities visually into modules or domains, like "Clients and Agents," "Property and Location," and "Transactions and Appointments." This separation enhances readability and shows how different parts of the system are interconnected.
- **Clear Domain Boundaries:** For example, keeping all client-related entities together in one section and property-related entities in another will provide a clear visual understanding of how the data flows between these groups.

4. Add Multiplicity Details

- **Clarify Cardinalities:** Explicitly define the cardinality of each relationship to make it clear how many instances of one class can relate to instances of another class. This

helps to understand constraints, like one `Client` signing multiple `Contracts` or a `Property` belonging to a specific `Neighborhood`.

5. Revise Entity Definitions

- **Attributes Consistency:** Some entities in the original diagram might have inconsistent or redundant attributes. It's essential to standardize attribute naming and ensure that each attribute only exists in the most relevant class. For example, ensure that `Location` data (like latitude and longitude) is strictly part of the `Location` class and not duplicated elsewhere.
- **More Descriptive Attribute Names:** Use more descriptive names for attributes to clarify their purpose. For example, instead of a generic `value`, you might have `propertyValue` to specify it belongs to the `Property`.

6. Use of Composition for Strong Relationships

- **Property and Location:** The relationship between `Property` and `Location` should use composition since a `Location` is fundamentally tied to the existence of a `Property`. This indicates that a `Location` cannot exist independently of the `Property`.
- **Transaction History:** For handling transactions, using a `TransactionHistory` class linked to `Transaction` might better model multiple transaction records, avoiding confusion in a one-to-many relationship.

7. Refine Role-Based Relationships

- **Agent and Client Interactions:** Clarify the role of the `Agent` in managing appointments and transactions. Clearly distinguish between what activities an `Agent` initiates versus what a `Client` or `Property` is involved in.
- **Manage Client-Property Link:** Reconsider how `Clients` interact with `Properties`—whether they only hold viewings (appointments) or if they have ownership records.

Conceptual Changes Summary

The conceptual improvements revolve around:

- **Reducing clutter** by organizing the relationships more logically.
- **Strengthening associations** using UML principles like composition and inheritance.
- **Creating modular structures** for better separation of concerns.
- **Clarifying multiplicity** to specify the nature of each relationship.
- **Standardizing and simplifying attributes** to remove redundancy.

These changes would lead to a more intuitive and readable diagram, helping both developers and stakeholders understand the system architecture more clearly.

After this prompt, ChatGPT provided several useful tips that helped refine the UML class diagram. It emphasized the importance of reducing clutter by organizing relationships and using aggregation or composition effectively to represent stronger associations between entities. We think these recommendations were helpful in improving the clarity and readability of the diagram.

A notable point was its recommendation in Step 6 to introduce a TransactionHistory class. This idea was a valuable addition to our UML diagram, as it provided a cleaner way to manage multiple transaction records, avoiding confusion in one-to-many relationships. This enhancement added depth to our model.

Based on these tips, we concluded that the revised UML diagram now aligns more closely with best practices.